

Ircama / ELM327-emulator

Q

+ ▾

📁

<> Code

🔍 Issues 17

🔗 Pull requests 1

🛠 Agents

🔄 Actions

🛡 Security and quality

📊 Insights

👁 Watch 24 ▾

🔗

☆

▾

ELM327 Emulator for testing software interfacing OBDII via ELM327 adapter supporting multi-ECU simulation

🔗 View license

🔗 Contributing

🔗 Security policy

☆ 633 stars

🔗 118 forks

👁 24 watching

🔗 Branches

📁 Activity

📁 Tags

🌐 Public repository

🔗 ma... ▾

🔗 1 Branch

📁 13 Tags

🔗

📁

🔍 Go to file

T

Go to file

Add file ▾

Code

⋮

	Odometer and supported PID fixes (#55)	7387317 · 3 months ago	🕒
📁 .github/workflows	More reliable behavior when the emulator ...		last year
📁 .jekyll-gh-pages	Add usage command and allow running th...		2 years ago
📁 elm	Odometer and supported PID fixes (#55)		3 months ago
📁 obd_dictionary	V3.0.0 - Support of UDS, tasks and other ...		5 years ago
📄 .gitignore	Add usage command and allow running th...		2 years ago
📄 CONTRIBUTING.md	V3.0.0 - Support of UDS, tasks and other ...		5 years ago
📄 LICENSE	Version 1.0.0		6 years ago
📄 MANIFEST.in	V3.0.0 - Support of UDS, tasks and other ...		5 years ago
📄 README.md	Add usage command and allow running th...		2 years ago
📄 RELEASES.md	Version upgrade to 1.0.2		6 years ago
📄 SECURITY.md	V3.0.0 - Support of UDS, tasks and other ...		5 years ago
📄 setup.py	Update pyreadline3 dependency for Pyth...		6 months ago

📖 README

🔗 Contributing

🔗 License

🔗 Security

✎

ELM327-emulator

📦 pypi v3.0.5

🐍 python 3

📄 downloads 364/month

📄 license CC-BY-NC-SA-4.0

A Python emulator of the ELM327 OBD-II adapter connected to a vehicle supporting multi-ECU simulation.

ELM327-emulator connects client applications to multiple emulated [ECUs](#) via [OBD-II](#) interface through different networking systems, including serial communication (where [pseudo-terminals](#) are used if supported by the operating systems), or direct interaction with communication devices, or TCP/IP, or Bluetooth. The software simulates an [ELM327](#) adapter connected to a vehicle, includes a command-line interface for extensive monitoring and offers a documented Python development framework to implement ECU emulation objects.

ELM327-emulator is able to support basic ELM327 commands and OBD service requests through stateless request/response method via OBD-II, but can also handle stateful [UDS](#) communication with [ISO-TP](#) Flow Control and [Keyword Protocol 2000](#), concurrently emulating multiple ECUs. It is designed to be extended via a plugin architecture to allow easy development of specific tasks implementing workflows, including the possibility to simulate anomalies for testing purposes. Many AT commands are supported, as well as some [OBDLink](#) AT/ST [commands](#).

ELM327-emulator supports different operating systems including Windows, macOS and UNIX/Linux; it is agnostic of the client application and has been tested with [python-OBd](#) as well as with many applications on Windows, Linux and on smartphone devices.

An internal dictionary (named *ObdMessage*) allows configuring the emulation, which is currently set to reproduce the message flow generated by more ECUs, including the basic communication of a Toyota Auris Hybrid car (through the `scenario car` option) and can be easily extended to simulate communication produced by other vehicles or ECUs. In case of more complex interactions (e.g., entering privileged diagnostic sessions and performing reset or flash upload operations), the dictionary can instance tasks, which are implemented through plugins, very simple to be developed by users extending the software. Some example of plugins is already included.

ELM327-emulator also offers a feature that compares the emulation with the direct connection of an OBD-II adapter to allow extending the dictionary by reporting unmatched requests. In addition, it provides an auxiliary scanning tool (*obd_dictionary*) that builds the *ObdMessage* dictionary of a specific vehicle by automatically querying all standard OBD service requests one by one (as well as querying additional custom PIDs specified by the user via CSV file); the dictionary can then be used to emulate stateless request/responses of a specific car or ECU.

Installation

Check that the [Python](#) version is 3.6 or higher (`python3 -V`), then install *ELM327-emulator* with the following command:

```
python3 -m pip install ELM327-emulator
```



This is enough to run the software.

Prerequisite components: *pyyaml*, *python-daemon*, *obd*; in addition, with Windows also *pyreadline3*. All needed prerequisites are automatically installed with the package.

obd (*python-OBd*) is needed for *obd_dictionary*. It is better to use an updated version of *python-OBd* package (e.g., the one installed from GitHub with `python3 -m pip install --upgrade git+https://github.com/brendan-w/python-OBd.git`).

Alternatively to the above mentioned installation method, the following steps allow installing the latest version from GitHub.

- Optional preliminary configuration with Ubuntu (if not already done):

```
sudo apt-get update
sudo apt-get -y upgrade
sudo add-apt-repository universe # this is only needed if "sudo apt install python3-pip" fails
sudo apt-get update
sudo apt install -y python3-pip
python3 -m pip install --upgrade pip
sudo apt install -y git
```



- Optional preliminary configuration with Windows:

- install the latest version of [Python](#) (also available from [Microsoft Store](#));
- if you want to install *ELM327-emulator* from GitHub, install *git* from [Git-scm](#) or using the [Git for Windows installer](#);
- if the interface to use is a COM port (e.g., not TCP/IP or Bluetooth), also install [com0com](#) (no installation is needed when using TCP/IP or Bluetooth interfaces);
- optionally, check that PIP is upgraded (`python3 -m pip install --upgrade pip`).

- Run this command:

```
python3 -m pip install git+https://github.com/ircama/ELM327-emulator
```



To uninstall:

```
python3 -m pip uninstall -y ELM327-emulator
```



Basic use

The emulator allows batch and interactive mode. The latter is the default and can be executed as follows:

```
python3 -m elm
```



or simply:

```
elm
```



After starting the program, the emulator is ready to use.

To enable the preconfigured set of OBD service requests of a Toyota Auris Hybrid car, enter `scenario car` (or, alternatively, run the emulator with the `-s car` option, i.e. `python3 -m elm -s car`).

By default, *ELM327-emulator* uses serial communication. The external application interfacing the emulator just needs to connect to the virtual device shown by the emulator and interact with the vehicle as if it was accessing a real ELM327 adapter.

Alternatively to the serial communication, *ELM327-emulator* supports TCP/IP networking through the `-n` option, followed by the port number (which in most cases is 35000). Example:

```
python3 -m elm -n 35000
```



All subsequent information is not needed for basic usage of the tool and allows mastering *ELM327-emulator*, exploiting it to test specific features including the simulation of communication exceptions, which are not always easy to be reproduced with a real link.

Running the pre-built executable program

The *elm327-emulator.zip* archive in the [Releases](#) folder includes the *elm327-emulator.exe* executable asset; the ZIP archive is auto-generated by a [GitHub Action](#). *elm327-emulator.exe* is a Windows application that can be directly executed.

Compatibility

ELM327-emulator has been tested with Python 3.6, 3.7, 3.8, 3.9. Previous Python versions are not supported.

When using serial communication, with UNIX/Linux OSs, this code uses `pty` pseudo-terminals. With Windows, you should first install [com0com](#) (a kernel-mode virtual serial port driver), or [other virtual serial port software](#); alternatively, [cygwin](#) and [Windows Subsystem for Linux](#) (WSL) are supported.

Usage

The description of the *ELM327-emulator* command-line option is the following:

```
usage: elm [-h] [-V] [-e] [-l] [-t] [-d] [-b FILE] [-p PORT] [-P DEVICE_PORT] [-a BAUDRATE] [-v LOG] [-s SCENARIO]
[-n INET_PORT]
        [-H INET_FORWARD_HOST] [-N INET_FORWARD_PORT] [-S FORWARD_SERIAL_PORT] [-B FORWARD_SERIAL_BAUDRATE] [-T
FORWARD_TIMEOUT]

optional arguments:
  -h, --help            show this help message and exit
  -V, --version          Print ELM327-emulator version and exit
  -e, --no-echo          Disable echo by default
  -l, --newline          Use newline (<NL>) instead of carriage return <CR> for detecting a line separator
  -t, --terminate        Terminate the daemon process sending SIGTERM
  -d, --daemon           Run ELM327-emulator in daemon mode.
  -b FILE, --batch FILE Run ELM327-emulator in batch mode. Argument is the output file. The first line in that
file will be the virtual
                        serial device
  -p PORT, --port PORT   Set a serial communication port instead of using a pseudo-tty.
  -P DEVICE_PORT, --device DEVICE_PORT
                        Set the communication device to be opened instead of using a pseudo-tty port.
  -a BAUDRATE, --baudrate BAUDRATE
                        Set the serial device baud rate used by ELM327-emulator.
  -v LOG, --log LOG      Preset a log level in interactive mode.
  -s SCENARIO, --scenario SCENARIO
                        Set the scenario used by ELM327-emulator.
  -n INET_PORT, --net INET_PORT
                        Set the INET socket port used by ELM327-emulator.
  -H INET_FORWARD_HOST, --forward_host INET_FORWARD_HOST
                        Set the INET host used by ELM327-emulator when forwarding the client interaction to a
remote OBD-II port.
  -N INET_FORWARD_PORT, --forward_port INET_FORWARD_PORT
                        Set the INET socket port used by ELM327-emulator when forwarding the client interaction to
a remote OBD-II port.
  -S FORWARD_SERIAL_PORT, --forward_serial_port FORWARD_SERIAL_PORT
                        Set the serial device port used by ELM327-emulator when forwarding the client interaction
to a serial device.
  -B FORWARD_SERIAL_BAUDRATE, --forward_serial_baudrate FORWARD_SERIAL_BAUDRATE
                        Set the device baud rate used by ELM327-emulator when forwarding the client interaction to
a serial device.
  -T FORWARD_TIMEOUT, --forward_timeout FORWARD_TIMEOUT
                        Set forward timeout as floating number (default is 5 seconds).

ELM327-emulator v3.0.0 - ELM327 OBD-II adapter emulator
```

Description

The communication port to be used by the application interfacing the emulator is displayed when starting the program. E.g. on UNIX/Linux:

```
ELM327-emulator is running on /dev/pts/0
```

When running on Windows, the following message is shown:

```
ELM327-emulator is running on com0com serial port pair reading from COM3
```

Embedded dictionary of AT Commands and OBD service requests

A [dictionary](#) named *ObdMessage* is used to define commands and PIDs. The dictionary includes more sections (named scenarios):

- 'AT' : set of default AT and ST commands
- 'default' : set of default PIDs
- 'car' : PIDs of a Toyota Auris Hybrid vehicle
- 'mt05' : basic set of PIDs of an MT05 ECU from Delphi used in many motorbikes and ATV's
- any additional custom section can be used to define specific scenarios

Default settings include both the 'AT' and the 'default' scenarios.

The dictionary used to parse each ELM command is dynamically built as a union of three defined scenarios in the following order: 'default', 'AT', custom scenario (when applied). Each subsequent scenario redefines the commands of the previous scenarios. In principle, 'AT' scenario is added to 'default' and, if a custom scenario is used, this is also added on top, and all equal keys are replaced. Then the Priority key defines the precedence to match elements.

If a custom scenario is selected through the *scenario* command, any key defined in the custom scenario replaces the default settings ('AT' and 'default' scenarios).

The key used in the dictionary consists of a unique identifier for each PID. Allowed case-insensitive values for each key (PID):

- 'Request' : received data; a [regular expression](#) can be used
- 'Descr' : string describing the PID
- 'Exec' : command to be executed
- 'Log' : *logging.debug* argument
- 'Info' : *logging.info* argument
- 'Warning' : *logging.warning* argument
- 'ResponseFooter' : run a function and returns a footer to the response (a [lambda function](#) can be used)
- 'ResponseHeader' : run a function and returns a header to the response (a [lambda function](#) can be used)
- 'Response' : returned data; can be a string, or a list/tuple of strings; if more strings are included (e.g., if a tuple or a list is used), the emulator selects one of them each time, according to the criteria defined by the `choose` command, which can be either sequential (default) or random.
- 'Action' : can be set to 'skip' in order to skip the processing of the PID
- 'Header' : if set, process the command only if the corresponding header (CAN ID) matches
- 'Priority'=number : when set, the key has higher priority than the default (highest number = 1, lowest = 10 = default)
- 'Task' : if set, the related header/request activates a specific task referring to an installed plugin.

The emulator provides a monitoring front-end, supporting commands and controlling the backend thread which executes the actual process.

Built-in keywords

At the `CMD>` prompt, the emulator accepts the commands described in the following table.

Command	Description
help	List available commands (or detailed help with "help cmd").
port	Print the used TCP/IP port, or the used device, or the serial COM port, or the serial pseudo-tty, depending on the selected interface.
test	Test the OBD-II request specified in the argument. Check also "verify" and "write". The autocompletion (by pressing or double-pressing TAB) allows prompting all defined OBD-II requests (PIDs).
write	Write the formatted XML response specified in the argument to the connected application. The ASCII null character is represented as <code>\x00</code> (Use "verify" to avoid the write operation.)
verify	Test the processing of the formatted XML response specified in the argument (like "write", but without writing to the application). The ASCII null character is represented as <code>\x00</code> .
loglevel	If an argument is given, set the logging level, otherwise show the current one. Valid numbers or words: CRITICAL=50, ERROR=40, WARNING=30, INFO=20, DEBUG=10. The autocompletion (by pressing or double-pressing TAB) allows prompting all available values.
quit	quit the program (or end-of-file/Control-D, or break/Control-C)
counters	print the number of each executed PIDs (upper case names), the values associated to some 'AT' PIDs (<i>cmd_...</i>), the unknown requests, the emulator response delay, the total number of executed commands (<i>commands</i>) and the current scenario (<i>scenario</i>). The related dictionary is <code>emulator.counters</code> .
edit	Edit a PID answer. Arguments: PID, position, replaced bytes. If only the PID is given, remove a previous editing.
tasks	Print all available plugins; for each used ECU, print all active tasks and dump related namespaces; dump also the shared namespaces.
pause	pause the execution. (Related attribute is <code>emulator.threadState = emulator.THREAD.PAUSED</code> .)

Command	Description
prompt	toggle prompt off/on if no argument is used, or change the prompt if using an argument
resume	resume the execution after pausing; also prints the used device. (Related attribute is <code>emulator.threadState = emulator.THREAD.ACTIVE</code>)
delay <n>	delay each emulator response of <n> seconds (floating-point number; default is 0.5 seconds)
wait <n>	delay the execution of the next command of <n> seconds (floating-point number; default is 10 seconds)
timer [<name> <value>]	Print or set the UDS timers P1, P2, P3, P4. The first argument is the timer name, the second is the value in seconds. Without arguments, print all timer values. Decimals are allowed.
engineoff	switch to <i>engineoff</i> scenario
scenario <scenario>	switch to <scenario> scenario; if the scenario is missing or invalid, defaults to 'car' . The autocompletion (by pressing or double-pressing TAB) allows prompting all compatible scenarios. defined in <code>emulator.ObdMessage</code> . (Related attribute is <code>emulator.scenario</code> .)
default	reset to <i>default</i> scenario
reset	reset the emulator (counters and variables)
color	toggle usage of colors off/on
history [<n>]	print the last 20 items of the command history; if an argument is given, print the last n items in the history; with argument <i>clear</i> , clear the history. The command history is permanently saved to file <i>.ELM327_emulator_history</i> within the home directory.
merge <module>	import a scenario from an external module and merges it with the emulator configuration. <module> shall be a Python file including the <code>ObdMessage</code> dictionary (e.g., generated by <i>obd_dictionary</i>) without <i>.py</i> extension (notice that the physical file shall be in the current directory and shall end with <i>.py</i>). The autocompletion allows prompting all compatible files in the current directory (type <i>merge</i> , then press space, then press TAB). After a successful merge, the new scenario can be activated through the <i>scenario</i> command.
version	Print ELM327-emulator version. With an argument, set the ELM version. If the argument is <i>hexheader</i> followed by a sequence of hex digits, the header of the ELM version is updated with the sequence. If the argument is <i>reset</i> , header and ELM version strings are set to default values. The autocompletion can be used with this command.
commands	List the description of each available command.
choice	Print or select the adopted method to choose the return value of answers that are expressed as a list of data. Possible values are "sequential" (the returned value follows the list sequence, which is the default mode) or "random" (the returned value is randomly selected within the values in the list). Optional list of weights can be added; decimals are allowed; the default value is 1. Autocompletion is allowed for this command.
usage	Open a web browser showing this documentation.

In addition to the previously listed keywords, any Python command is allowed to query/configure the backend thread.

Examples of usage of the `choice` command:

```
# Use sequential mode with default weight:
choice SEQUENTIAL

# Use random mode with default weight:
choice RANDOM

# Use sequential mode with weight 30 (each step occurs after 30 requests):
choice SEQUENTIAL 10

# Use sequential mode with weights [10, 1, 0.5].
# Digits represent the relative probability of respective values in the "Response" list.
# Missing weights are replaced with 1. In the example,
# the first value in the list is 10 times more likely to be selected than the second,
# which is twice the third; this third is half the fourth and all the subsequent values.
choice RANDOM 10 1 0.5
```



At the command prompt, cursors and [keyboard shortcuts](#) are allowed. Autocompletion (via TAB key) is active for all previously described commands and also allows Python keywords and namespaces (built-ins, self and global). If the autocompletion matches a single item, this is immediately expanded; Conversely, if more possibilities are matched, none of them is returned, but pressing TAB again a list of available options is displayed.

Testing OBD-II requests

ELM327-emulator allows testing OBD-II requests through the *test* command directly via the command line, like in this example, where the '010C' pid is tested:

```
python3 -m elm -s car
test 010c
test ath1
test 010c
```



The autocompletion is allowed for the argument, to prompt and select values (PIDs) defined in the dictionary.

The answer (*Command output*) will be `41 0C 13 FB \r\r> , 0K\r\r> , 7E8 04 41 0C 09 F6 \r\r> ,` which will reflect what *ELM327-emulator* returns to a real OBD-II application.

Special setters

The counters starting with *cmd_...* are special setters. They are represented in the following table and store data related to AT/ST commands.

Special setter	Related AT/ST command	Handled	Description	PID
cmd_echo	ATE0, ATE1	Yes	Echo off or on	AT_ECHO
cmd_linefeeds	ATL0, ATL1	Yes	Linefeeds off or on	AT_LINEFEEDS
cmd_spaces	ATS0, ATS1	Yes	Spaces off or on	AT_SPACES
cmd_set_header	ATSH	Yes	Set the request header	AT_SET_HEADER
cmd_use_header	ATH0, ATH1	Yes	Headers off or on	AT_USE_HEADERS
cmd_last_cmd		Yes	Last executed request	(ref. <i>cmd_last_pid</i>)
cmd_last_pid	(ref. <i>cmd_last_cmd</i>)	Yes	Label of the last used PID	(last PID)
cmd_cra	ATCRAx (x=hex digit)	Yes	Set SET CAN Receive address filter argument	AT_SET_CAN_RX_ADDR
cmd_caf	ATCAF0, ATCAF1	Yes	Set CAN Auto formatting on/off (0 = use PCI byte in requests)	AT_CAF
cmd_cfc	ATCFC0, ATCFC1	Yes	Set CAN Flow control off or on	AT_CFC
cmd_can	(none)	Yes	Include headers in CAN requests (experimental - to be removed)	(none)
cmd_atkw	ATKW, ATKW0, ATKW1	Yes	Display or set keyword	AT_DKW, AT_SKW
cmd_version	ATI, ATWS, ATZ	Yes	ELM version (def. <i>ELM327 v1.5</i>)	AT_I, AT_WARM_START, AT_RESET
cmd_fcsM	ATFCSM m (m = mode)	No	Set flow control to mode m	AT_FCSM
cmd_proto	ATSPx (x=hex digit)	No	Set protocol	AT_PROTO
cmd_memory	ATM0, ATM1	No	Set Memory off or on	AT_MEMORY
cmd_fcsh	ATFCSHx (x=hex digit)	No	Set FLOW CONTROL set HEADER	AT_FCSh

Special setter	Related AT/ST command	Handled	Description	PID
cmd_fcscd	ATFCSDx (x=hex digit)	No	Set FLOW CONTROL set DATA	AT_FCSD
cmd_fcsm	ATFCSMx (x=hex digit)	No	Set FLOW CONTROL set MODE	AT_FCSM
cmd_timeout	ATSTx (x=hex digit)	No	Set timeout	AT_SET_TIMEOUT
cmd_cea	ATCEAx (x=hex digit)	No	Set CAN extended address	AT_CEA
cmd_adaptive_t	ATATx (x=0, 1, 2)	No	Set adaptive timing mode	AT_ADAPTIVE_TIMING
cmd_try_proto	ATTPx (x=hex digit)	No	Try protocol	AT_TRY_PROTO
cmd_iso_baud	ATIB 10, or 48, or 96	No	Set ISO baud rate	AT_ISO_BAUD
cmd_response	ATRO, ATR1	No	Set responses off/on	AT_RESPONSES
cmd_brd	ATBRDn (n=two digits)	No	Set UART baud rate divisor	AT_BRD
cmd_long_msg	ATAL, ATNL	No	Set message length	AT_LONG_MSG, AT_NORMAL_LENGTH
cmd_iaa	ATIIA hh (hh = addr)	No	Set the ISO 5-baud init address to hh	AT_ISO_INIT_ADDR
cmd_rec_addr	ATSR hh (hh = addr)	No	Set receive address	AT_SET_RECEIVE_ADDR
cmd_hfm	ATCM m (m = addr)	No	Set the CAN hardware filter mask	AT_CAN_HFM
cmd_brt	ATBRT t (t = timeout)	No	Set UART baud rate timeout	AT_BRT
cmd_wakeup	ATSW hh (hh = addr)	No	Set wakeup	AT_WAKEUP
cmd_st_slx	STSLXm (m=string)	No	Enable or disable sleep/wakeup triggers	ST_SLX
cmd_st_proto	STPp (p=number)	No	Set current ST protocol	ST_PROTO
cmd_st_fcap	STCFCPA t,STCAFCP t	No	Add a flow control address pair	ST_STCAFCP
cmd_stip4	STIP4 n (n = delay)	No	Set Tx Interbyte delay	ST_IP4
cmd_stpto	STPTO t (t = timeout)	No	Set OBD Request Timeout	ST_PTO
cmd_atv	ATV0, ATV1	No	Set variable DLC on or off	AT_V

Unhandled setter means that the AT/ST command is recognized, the related counter is valued but no process is currently associated.

echo and *linefeed* settings are both disabled by default. They can be configured via related AT commands (*ATE1* and *ATL1*). The special setters `cmd_echo` and `cmd_linefeeds` allow enabling them via command line. Example:

```
emulator.counters['cmd_linefeeds'] = True; emulator.counters['cmd_echo'] = True
```



This is the same as:

```
test ATL1
test ATE1
```



Possible values of `emulator.counters['cmd_linefeeds']` :

Value	Behaviour	separator among	separator at the end of
-------	-----------	-----------------	-------------------------

		lines	the response
0	(Default) Each line in the response is separated by one CR and the response is closed by two CRs (selected with ATL0).	\r	\r\r
1	Each line in the response is separated by one CR+LF and the response is closed by two CR+LF (selected with ATL1).	\r\n	\r\n\r\n
2	Each line in the response is separated by one LF and the response is closed by two LFs.	\n	\n\n
3	Each line in the response is separated and closed by a CR.	\r	\r
4	Each line in the response is separated and closed by a CR+LF.	\r\n	\r\n
5	Each line in the response is closed by a LF.	\n	\n

Space characters are inserted by default in the ECU response as per specification. To remove them, use the AT command *ATS0* or `emulator.counters['cmd_spaces'] = 0`.

By default the header (CAN ID) is not included in the ECU response. To add it, use the AT command *ATH1* or `emulator.counters['cmd_use_header'] = True`.

The default ECU header is ECU_ADDR_E (e.g., "7E0", producing answer "7E8"; ref. *obd_message.py*). Use `cmd_set_header` to customize it, or, alternatively use the command `test atsh <header>`.

The last executed request is stored in `cmd_last_cmd`. This is used to repeat the command when the 'fast' option is set (command repetition, using a newline). While executing the command, `cmd_last_pid` is valued with the PID label referred to the request.

Each time the interface is reset by an *ATZ* command, the special setters are restored to their default settings and any specific customization needs to be issued again. Use `emulator.presets` in order to preset the special setters so that they are applied as default values each time the interface is opened by an application. Example:

```
emulator.presets = { 'cmd_linefeeds': 4, 'cmd_spaces': 0 }
```



The *cmd_spaces* setting is the same as:

```
test ATS1
```



cmd_cra keeps the value set by *ATCRA* when setting and resetting the receive address filter. The following metacharacters are accepted:

- *X* for any single hex digit,
- *W* for any sequence of hex digits (one or more).

Example.

```
test ATCRA 7X8
test 0100
counters
```



It returns `cmd_cra` = 7X8.

To switch off autoformatting, set *cmd_caf* to *False*, like in the following example:

```
test atcaf0
counters
test 02 01 0C
```



To test the processing of full CAN messages, set *cmd_can* as in the following example:

```
emulator.counters["cmd_can"] = True
```



The above setting also implies the following ones, which are automatically changed by each subsequent CAN request:

```
self.counters['cmd_set_header'] # using the first three hex digits of the request
self.counters['cmd_caf'] = False
self.counters['cmd_use_header'] = True
```

Example of usage of `cmd_can`:

```
scenario car
emulator.counters["cmd_can"] = True
test 7E0 02 01 0C
test 7e2 02 01 5b
test 7E5 06 27 12 B1 51 D5 8F
```

To change the ELM version string in ELM commands, use the `version` command, which defaults to *ELM327 v1.5*. Use `version reset` to return to the default values of the ELM version string and related header. Check also the "*elm_version*" setter, which stores the temporary version string within the current session.

Example:

```
version ELM327 v2.3
test ati # Get the ELM327 version string
```

The returned code will be `'ELM327 v2.3\r\r>'`.

Through the *hexheader* argument of the `version` command, *ELM327-emulator* also allows to change the header bytes of the version string, which by default are two carriage return characters. The updated string should be composed by a sequence of hex digits. Example:

```
version hexheader 0d fc 0d 0d
```

Any change to the ELM version string and ELM header version bytes can be restored to default with:

```
version reset
```

Without parameters, `version` returns the *ELM327-emulator* version and ELM version parameters in use.

Other example to reproduce the ATZ output of a ScanTool.net OBDLink SX device:

```
version hexheader 00 00 00 00 0d 0d
version ELM327 v1.4b
test atz
```

The returned output is:

```
_____Raw command:_____
'<writeln>\x00\x00\x00\x00\r\rELM327 v1.4b</writeln>'

_____Command output:_____
'\x00\x00\x00\x00\r\rELM327 v1.4b\r\r>'
```

The output is then written to the application when using the following command:

```
write <writeln>\x00\x00\x00\x00\r\rELM327 v1.4b</writeln>
```

Editing answers on the fly

Static answers in the `obdMessage` dictionary can be edited on the fly through the `edit` command. It extracts the answer from a PID, stores it into the `emulator.answer` dictionary described in the next chapter and performs the editing in its data part. If the command is called with just the PID argument, it resets the `emulator.answer` dictionary for the referred PID, returning to the default answer.

Syntax: `edit PID <position> <replaced bytes>`

PID is an element name in the `ObdMessage` dictionary for the current scenario and with an associated static `'Response'` .

Position is a number starting from 0, which can be decimal (e.g., `edit VIN 10 AA BB CC DD`) or in any other format (`0o` for octal, `0x` for hex, `0b` for binary; example: `edit VIN 0x0A AA BB CC DD`).

Replaced bytes is a string of hexadecimal digits in any format.

Note: this command only works for PIDs which have a static `'Response'` in the `ObdMessage` dictionary. `'ResponseHeader'` , `'ResponseFooter'` and `'Task'` are not supported.

For more complex actions, the `Edit` class can be used with its `answer()` method, also allowing the Context Manager. See the related comments in *interpreter.py* for details on how to use this class.

The following example sets an edited response for 15 seconds, then restores the default answer.

```
with Edit(emulator, 'VIN') as e: e.answer(10, "AA BB CC DD"); time.sleep(15)
```



The following is a more complex command where the answer changes every three seconds:

```
with Edit(emulator, 'VIN') as e: \n\t for i in ['AA' 'BB' 'CC' 'DD']: \n\t\t e.answer(10, "01 02 03 " + i) \n\t\t tir
```



The `edit` command uses PIDs in the current scenario. To change scenario, use the `scenario` command. Example to change the value of FCLCINT1 (C/L Fuel Corr Int Cyl 1) on the MONITOR pid (2101, Delco "Mode 1 Message") in the "mt05" scenario (Delphi MT05 ECU) to "AA BB" (spaces between hex digits are optional):

```
scenario mt05
edit MONITOR 0x0E AA BB
```



Notice that the mt05 scenario is automatically set by the 'UDS_START_COMM' PID (81). The method to dynamically change scenario is `self.set_sorted_obd_msg(scenario)` .

Data Link and Network Layers

ELM327-emulator includes a basic processing of the ISO 15765-2 ISO-TP Layer and KWP2000 ISO 14230-2:1999 Data Link layer. The following elements are implemented:

- 3-byte and 4-byte KWP2000 header with frame length management (the "Format" byte always assumes address information and physical addressing; other cases are ignored),
- ISO-TP 11 bit CAN identifier (29 bit CAN identifiers are not supported)
- ISO-TP Single frame (SF), First frame (FF), Consecutive frame (CF), Flow control frame (FC),
- Basic input flow control of ISO-TP (with generation of FC output frames); output flow control (handling of FC input frames) is ignored,
- KWP2000 Checksum byte (CS) at the end of the ISO 14230-2:1999 message block (checksum verification in requests and checksum generation in responses),
- ISO-TP P1, P2, P3 and P4 [timers](#).

The KWP2000 format is detected by a header `>=` three bytes.

Advanced usage

ELM327-emulator allows changing the UDS P1, P2, P3 and P4 [timers](#) via the `timer` command. The P4 timer controls the max delay between each entered character and by default is not active (e.g., set to 1440 seconds). The P4 timer can either be configured via `timer P4 value` , or by setting `emulator.counters['req_timeout']` . Decimals are allowed. Some adapters set P4 by default, discarding characters if each of them is not entered within a short time limit (apart from the first one after a CR/Carriage Return). The appropriate emulation for this timeout is to set `emulator.counters['req_timeout']=0.015` (e.g., 15 milliseconds). Typing commands by hand via terminal emulator with such adapters is not possible as the allowed timing is too short. The same happens when setting `req_timeout` to 0.015 (or `timer P4 0.015`).

The command prompt also allows configuring the `emulator.answer` dictionary (ref. also previous paragraph), which has the goal to dynamically redefine answers for specific PIDs (`'Pid': '...'`). Its syntax is:

```
emulator.answer = { 'pid' : 'answer', 'pid' : 'answer', ... }
```



Example:

```
emulator.answer = { 'SPEED': '<writeln>NO DATA</writeln>', 'RPM': '<writeln>NO DATA</writeln>' }
# Or, alternatively:
emulator.answer['SPEED']='<writeln>NO DATA</writeln>'
emulator.answer['RPM']='<writeln>NO DATA</writeln>'
test 010d
test 010c
```



The above example forces SPEED and RPM PIDs to always return "NO DATA".

To reset the *emulator.answer* string to its default value:

```
emulator.answer = {}
# Or, alternatively:
del emulator.answer['SPEED']
del emulator.answer['RPM']
test 010d
test 010c
```



To simulate that the adapter is not connected to the vehicle:

```
emulator.answer['AT_R_VOLT'] = '<writeln>0.0V</writeln>'
test atrv
```



As mentioned in the previous paragraph, the `edit` command simplifies the usage of *emulator.answer*.

The *emulator.ELM_R_UNKNOWN* parameter allows customizing the message returned in case of unknown/invalid command. The default message is `?\r`, with an addition of a trailing `\r`. This message can be customized; for example, to just get `\r`, set the following:

```
emulator.ELM_R_UNKNOWN = ''
```



The dictionary can be used to modify answers within a workflow. The front-end allows implementing basic Python workflows and, when used in batch mode, can also be controlled by a piped external supervisor. The following examples show some simple workflows in interactive mode.

Example of automation which suspends the emulator for 10 seconds:

```
emulator.threadState = emulator.THREAD.PAUSED; time.sleep(10); emulator.threadState = emulator.THREAD.ACTIVE
```



Example of an automation that simulates the off/on ignition states:

```
CMD> for i in range(10): emulator.scenario="car" if i % 2 else "engineoff"; print(emulator.scenario); time.sleep(10)
engineoff
car
engineoff
car
engineoff
car
engineoff
car
engineoff
car
```



Configuring response strings

Response, *ResponseHeader*, *ResponseFooter*, *emulator.answer* and *emulator.ELM_R_UNKNOWN* support the following XML tags (which shall be produced without the *xml* envelope):

Tag	Line separator	Behaviour	Helper function
<writeln>	Yes	The content is returned with the addition of a line separator at the end. <writeln></writeln> or <writeln /> means newline.	ST(writeln)
<space>	No	The content is returned with the addition of a space at the end. <space></space> or <space /> means adding a simple space.	
<string>	No	The content is returned with no space and no line separator at the end.	
<header> , <size> , <data>	Yes	Standard response format composed of the concatenation of hexadecimal ECU header, related size code and hexadecimal data with the addition of a line separator at the end. The <i>data</i> part is not automatically converted into a multiframe if longer than 7 bytes.	HD(header), SZ(size), DT(data)
<header> , <size> , <subd>	No	Standard response format composed of the concatenation of hexadecimal ECU header, related size code and hexadecimal data with no line separator at the end (same as before).	
<exec>	No	Execution of single or multiple in-line Python commands (expressions or statements) returning the expression evaluation with no line separator at the end; any previous string is printed before the execution.	
<eval>	No	Like <code>exec</code> , but any previous string is concatenated (not immediately printed).	
<rh>	No	Force the usage of the request header included in this tag to generate the response instead of the one included in the real request.	
<rd>	No	Force the usage of the request data included in this tag to generate the response instead of the one included in the real request.	
<flow>	Yes	Generate a flow control response.	
<answer>	Yes	Generate an UDS generic response basing on request header (including header, length and generic data including the string in within this tag). Automatically generate Single Frames, Flow control frames, First Frame and Consecutive Frames. It is up to the dictionary or task to generate the appropriate positive or negative answer bytes.	AW(answer)
<pos_answer>	Yes	Generate an UDS positive answer response basing on request header and request data (the generated response includes header, length and UDS positive response data, then adding the string within this tag). Automatically generate multiframes if needed. If <code><rh></code> is used, the default request header is replaced by the one in this tag. If <code><rd></code> is used, the default request data is replaced by this tag. See the table named <code>uds_sid_pos_answer</code> in elm.py to check how <i>ELM327-emulator</i> computes the number of bytes to add to the answer for each requested SID.	PA(pos_answer)
<neg_answer>	Yes	Generate an UDS negative answer response basing on request header and request data (the generated response includes header, length and UDS negative response data, then adding the string within this tag). Automatically generate multiframes if needed.	NA(neg_answer)

Strings among tags are allowed and are returned as they are, with no line separator and stripping blank heading and footing characters.

Whenever possible, the usage of *pos_answer* and *neg_answer* is suggested, totally relying on *ELM327-emulator* for the construction of the answer. In such cases, *ELM327-emulator* builds the final bytestream basing on [ISO 14230-2:1999](#) for the Data Link Layer and [ISO 14230-3:1999](#) for the Application Layer.

The `verify` command can be used to test an XML response: the returned message will show what in normal operation is outputted to the communication port. Notice that, in order to check the produced `<header>` and `<size>` bytes, `cmd_use_header` shall be set to *True* (e.g., `test_ath1`); these bytes are not returned when `cmd_use_header` is *False*. Besides, bytes included in the `<data>` tag will be returned with all spaces stripped out if `cmd_spaces` is *False* (e.g., `test_atsp0`).

Tag nesting is not allowed.

Unknown tags are skipped.

Some special characters must be escaped within the XML fields, like:

Symbol (name)	Escape Sequence
< (less-than)	< (or <)
> (greater-than)	> (or >)

The `exec` tag for instance can be used to embed real-time delays between strings or to differentiate answers. The return value of a statement is ignored. The evaluation of an expression is substituted. Example: `'Response' = '<string>SEARCHING...</string><exec>time.sleep(4.5)</exec><writeln /><writeln>UNABLE TO CONNECT</writeln>'`. Notice that, as `time.sleep` is a statement, the related return value is ignored.

Further processing can be achieved through a *lambda function* applied to `ResponseHeader`, `ResponseFooter`. It has to manage the following parameters: `self, cmd, pid, uc_val` (e.g., `lambda self, cmd, pid, uc_val:`).

- `cmd`: the request, received by the client application
- `pid`: the PID identifier (which can be used as key to index `self.counters` and `OBDMessage`)
- `uc_val`: `OBDMessage` dictionary related to `pid` with all keys converted to uppercase (e.g., `uc_val['RESPONSE']`).

Example of PID definition within the `OBDMessage` dictionary:

```
'ELM_PIDS_A': {
    'Request': '^0100$',
    'Descr': 'PIDS_A',
    'ResponseHeader': \
        lambda self, cmd, pid, uc_val: \
            '<string>SEARCHING...</string>'
            '<exec>time.sleep(4.5)</exec><writeln />'
            '<writeln>UNABLE TO CONNECT</writeln>' \
            if self.counters[pid] == 1 else \
            self.choice([ST('NO DATA'), ST('BUS INIT:ERROR')]),
    'Response': '',
    'Priority': 5
},
```



In the above example, the first time `ResponseHeader` is executed (`self.counters[pid] == 1`), the produced response is `SEARCHING...` , followed by a one-second delay and then `\rUNABLE TO CONNECT\r\r`. For all subsequent messages, the response will be different and produces either `NO DATA\r\r` or `BUS INIT:ERROR\r\r`, according to the criteria (sequential or random) defined by the `choice` command.

The ability to add dynamic differentiators and delays within responses enables testing specific use cases and exceptions that are difficult to be achieved through a real connection with a car. These not only apply to the `OBDMessage` dictionary (by editing `obd_message.py`), but also to `emulator.answer` and `emulator.ELM_R_UNKNOWN`, that can be configured through the command line. Consider for instance the following dynamic configuration via command line:

```
emulator.answer['SPEED'] = '<exec>ECU_R_ADDR_E + " 03 41 0D 0A " if randint(0, 100) > 20 else "NO DATA"</exec><write'
scenario car
test ath1
test atsh7e0
test 010D
```



In the above example, which illustrates an in-line expression substitution, the configuration of the 'SPEED' PID ([Vehicle speed](#)) is replaced with a dynamic answer and the 'SPEED' PID will return `7E8 03 41 0D 0A + line separator` for most of the time. With 20% probability, `NO DATA + line separator` is returned. Notice that the last line separator is common to both options.

The following example shows how to dynamically generate an answer via command line by converting decimal numbers to hex string in order to allow comfortable testing of a PID by specifying decimal input values. Suppose that the PID needs to double the input. We use `CUSTOM_FUEL_LEVEL` PID in the example, testing the answer related to 15.5 liters.

Preliminarily, test number conversion with the command line:

```
"%.2X" % int(15.5*2)
1F
```



Apply it to `CUSTOM_FUEL_LEVEL` PID so that it returns `7C8 03 61 29 1F \r` :

```
emulator.answer['CUSTOM_FUEL_LEVEL'] = '<header>7C8</header><size>03</size><subd>61 29</subd><eval>\"%.2X\" % int(15.5*2)</eval>'
scenario car
test ath1
test atsh7c0
test 2129
```

The output is:

```
'7C8 03 61 29 1F \r\r>'
```

Or, alternatively, use the header variable instead of the header digits:

```
emulator.answer['CUSTOM_FUEL_LEVEL'] = '<exec>ECU_R_ADDR_I + \" 03 61 29 \" + \"%.2X\" % int(15.5*2)</exec><writeln>'
</writeln>'
scenario car
test ath1
test atsh7c0
test 2129
```

The following command sets SPEED ([Vehicle speed](#)) to 60 km/h via command line (60 can be changed to any integer value between 0 and 255):

```
emulator.answer['SPEED'] = '<header>7E8</header><size>03</size><subd>41 0D</subd><eval>\"%.2X\" % 60</eval><space />'
<writeln />'
scenario car
test ath1
test atsh7e0
test 010D
```

The output is:

```
'7E8 03 41 0D 3C \r\r>'
```

The following command sets RPM ([Engine RPM](#)) to 500 via command line:

```
emulator.answer['RPM'] = '<exec>ECU_R_ADDR_E + \" 04 41 0C %.4X\" % int(4 * 500)</exec><writeln />'
```

or

```
emulator.answer['RPM'] = '<header>7E8</header><size>04</size><subd>41 0C</subd><eval>\"%.4X\" % int(4 * 500)</eval>'
<space /><writeln />'
scenario car
test ath1
test atsh7e0
test 010C
```

In both cases, the output is:

```
'7E8 04 41 0C 07D0 \r\r>'
```

To list the configuration, type `emulator.answer`, or simply `counters`. To remove the dynamic answer and return to the default configuration of the 'SPEED' PID, type `del emulator.answer['SPEED']`.

Command to configure PID '0100' answer (*PIDS_A*) to `BUS INIT: 0K` for its first query and to `48 6B 13 41 00 BE 1F B8 11 AD \r` for all the subsequent queries:

```

emulator.answer['ELM_PIDS_A'] = '<exec>"BUS INIT: OK" if self.counters["ELM_PIDS_A"] &lt; 2 else "48 6B 13 41 00
BE 1F B8 11 AD "</exec><writeln />'
scenario car
test ath1
test atsh7e0
emulator.counters["ELM_PIDS_A"] = 0
test 0100
test 0100

```

The output is:

```

'BUS INIT: OK\r\r>'
'48 6B 13 41 00 BE 1F B8 11 AD \r\r>'

```

The *ELM_PIDS_A* counter (`emulator.counters["ELM_PIDS_A"]`) can be reset with:

```

emulator.counters["ELM_PIDS_A"] = 0

```

The following example shows how to use the `verify` command (without single or double commas) to quickly test the conversion of an XML response; `write` does the same and also writes the produced output to the opened device.

```

python3 -m elm -s car
test ath1
test ats0
verify <header>7E0</header><size>03</size><data>01 02 03</data>

```

The output will be `'7E003010203\r\r>'`.

In the next example, we will use the *pos_answer* tag, that needs the request header and the request data to produce a valid UDS positive response; in normal operation, those data are automatically inserted by *ELM327-emulator* upon each request; if using `verify` (which has no clue about any previous request), they need to be specifically included (as cannot be acquired by the context). As mentioned, `test ath1` instructs *ELM327-emulator* to return header and length.

```

test ath1 # return header and length
verify <rh>7E0</rh><rd>0902</rd><pos_answer>01 53 42 31 5A 53 33 4A 45 36 30 45 32 38 32 31 30 32</pos_answer>

```

The result will be `7E8 10 14 49 02 01 53 42 31 \r7E8 21 5A 53 33 4A 45 36 30 \r7E8 22 45 32 38 32 31 30 32 \r\r> .`

The *answer* tag simply computes the length of the data bytes and adds the header; it does not need the *rd* tag; using the above example, to generate the same answer we need to add the UDS positive answer data "49 02":

```

test ath1
verify <rh>7E0</rh><answer>49 02 01 53 42 31 5A 53 33 4A 45 36 30 45 32 38 32 31 30 32</answer>

```

Then the result will be the same: `7E8 10 14 49 02 01 53 42 31 \r7E8 21 5A 53 33 4A 45 36 30 \r7E8 22 45 32 38 32 31 30 32 \r\r> .`

Example of *neg_answer* tag:

```

test ath1
verify <rh>7E0</rh><rd>010F</rd><neg_answer>44</neg_answer>

```

The result will be `7E8 03 7F 01 44 \r\r> .`

Example of *flow* tag:

```

test ath1
verify <flow>20 00</flow>

```

The result will be `7E8 30 30 20 00 \r\r> .`

To write the output of a `test` command to the application, copy its *Raw command* output and paste it to a `write` command.

Timers

The `timer` command allows showing or changing the UDS timers.

Values are in seconds (floating numbers are allowed).

Timer name	Description	Default value	Note
P1	Inter byte time for ECU response	0	This timer is implemented by adding a fixed delay to each outputted character. If set to a value different than 0, <i>ELM327-emulator</i> outputs characters one by one, adding the indicated delay value after each of them.
P2	Time between tester request and ECU response or two ECU responses	0	Same as the <code>delay</code> command: this timer is implemented by adding a fixed delay (the one indicated in the <code>timer P2</code> value) after receiving each request (including also AT/ST commands) and before computing the response.
P3	Time between end of ECU responses and start of new tester request	5	The related value controls the expiration timeout between two responses: if expiring within a multiframe or within an active task, the related operation is interrupted and the active tasks of the same ECU are removed, executing the <code>stop()</code> method.
P4	Inter byte time for tester request	1440	The related value controls the time between each received request character to keep the whole request valid. If exceeding the timeout, the request is discarded. Changing this value configures the req_timeout counter.

Tasks

ELM327-emulator provides an extendable plugin architecture defining *tasks*, which are entities allowing the implementation of stateless and stateful procedures, that can be nested and are chainable. Through plugins, *ELM327-emulator* offers a development framework to easily implement ECU emulation objects, which are able to manage persistent data within the same instance (its namespace) and within the same ECU (shared namespace). Tasks allow emulating multiple ECUs concurrently, where each ECU has its own data space.

In case of stateless requests/responses, an ECU function can be emulated through a simple configuration in the dictionary, without usage of a plugin. Alternatively, if stateful routines or more complex programming are needed, an ECU function leverages the implementation of *tasks*. The structure of tasks is designed to simplify the development of complex ECU functions, like for instance flash upload/download operations.

When *ELM327-emulator* starts, it enumerates all available plugins in its *plugins* subdirectory (*elm/plugins*). Each plugin defines an own task, named with the file name of the plugin. All the file names of the plugins must start with `task_`.

A task is invoked in the dictionary through the `'Task'` tag, that refers to the name of an installed plugin. If a request associating a task is matched (including *Request* and possibly *Header* tags), its related task is activated by instantiating the *Task* class of the invoked plugin. After startup and after processing the request, the task can either terminate or remain active; in the latter case, it receives all subsequent requests related to the same header, allowing to implement a dedicated communication flow within a dedicated namespace. All tasks (regardless they terminate or remain active) take also advantage of a shared namespace, common to all requests addressed to the same ECU (see [ECU Tasks](#) for further information).

Tasks are interrupted by the following conditions:

- task termination performed by the plugin itself after processing the request (e.g, returning a method with `False` or with `self.TASK.TERMINATE` as the second value of the return tuple);
- communication reset (e.g., communication disconnection, or "ATZ", or *reset* command);
- expiration of the [P3 timer](#).

Tasks and plugins can be monitored through the `tasks` [command](#).

Multiple tasks can be concurrently instanced with different ECU IDs.

All plugins shall implement a class named *Task* derived from the *Tasks* class.

In a plugin, at least the `run()` method should be implemented, overriding the default method of the *Tasks* class. Allowed methods:

- `def start(self, cmd, *_):` invoked to process the first request of a just created task (not required)
- `def stop(self, cmd, *_):` invoked to process a request before interrupting the task (not required). Default is to return `Tasks.RETURN.ERROR` (do nothing).
- `def run(self, cmd, *_):` invoked on any request after the first one; if `start()` is not implemented, `run()` is always invoked.

Task methods are called after processing the ISO-TP data link, so the request passed to the task methods includes the whole message, where the associated multiframe (consisting of multiple frames received from the communication port) is already assembled and interpreted into a single data string of consecutive hex values, without spaces. Tasks methods return an XML response string as the first value of the return tuple; this XML string will be subsequently processed by the embedded ISO-TP data link, generating the output strings then sent to the communication port.

A multiframe is internally processed as a special temporary task named 'ISO-TP request pending'. It can be monitored through the `tasks` command.

All methods return a tuple of three elements:

- an XML response string, which will be subsequently interpreted by the ISO-TP processor and then written to the client application; *None* means nothing to output;
- a boolean (*True* = `Tasks.RETURN.CONTINUE` , or *False* = `Tasks.RETURN.TERMINATE`), to indicate whether the task remains active or terminates;
- a request string (e.g., *cmd*) which will be processed by *ELM327-emulator* after the transmission of the XML response string; the value can be:
 - *None*, meaning no subsequent request string to be processed, or
 - the same unchanged request of the task method invocation, where the returned request is sent to the standard processing of its dictionary response elements (without re-executing the same task), or
 - a different data than *cmd* in the task method, so that a full reprocessing of the new request is done (including running a task if defined).

This third element of the return tuple allows a task to also act as a filter or preprocessor, that receives a request (*cmd*), accounts it, possibly transforms it and forwards it to the standard processor.

Special return values:

- `Task.RETURN.ANSWER(answer)` , or `(answer, Tasks.RETURN.TERMINATE, None)` : used with standard positive or negative answers, terminating the task (without any further request processing after the task is terminated);
- `Task.RETURN.PASSTHROUGH(cmd)` , or `(None, Tasks.RETURN.TERMINATE, cmd)` : if the task returns the same unchanged *cmd* in the request, it is generally used for pure pass-through, like performing some accounting in the task, or logging, and then terminating the task while sending the same request to the standard processing of its dictionary response elements; if the task changes the returned data, a full reprocessing of the request is done;
- `Task.RETURN.ERROR` , or `(None, Tasks.RETURN.TERMINATE, None)` : used for error conditions; no output written while terminating the task;
- `Task.RETURN.INCOMPLETE` , or `(None, Tasks.RETURN.CONTINUE, None)` , used to allow internal processing of the request, without producing output and keeping the task active, so that the same task will also process all subsequent input requests addressed to the same ECU, until the task is terminated.
- `Tasks.RETURN.TASK_CONTINUE(cmd)` , or `(None, Tasks.RETURN.CONTINUE, cmd)` used for instance in ECU Tasks in order to pass the request to the subsequent processor, by keeping the ECU Task active for all subsequent requests, with its ECU namespace.

Check the *Tasks* class in [elm.py](#) for a list of the available variables initialized by its `__init__()` method.

Task namespaces

A task can exploit its own namespace, which is related to a specific task instance, that can remain active for subsequent requests if handled with `Tasks.RETURN.CONTINUE` . A task can also use the shared namespace of the ECU, which is associated to any kind of request referring to the same ECU ID. Notice that requests do not necessarily need to invoke a `'Task'` to access the ECU shared namespace: also `'ResponseFooter'` and `'ResponseHeader'` can reference the ECU shared namespace.

Other than storing local variables, the task namespace is useful to persist class properties if the task terminates with `Tasks.RETURN.CONTINUE` , and also to perform preprocessing through the related task methods (`start()` , `stop()` , `run()`), so that any subsequent request of the same ECU will be processed by the same task, until task termination. All subsequent calls of an active task share the same namespace. For instance, if a task is configured as a filter, its namespace can be used while preprocessing all subsequent requests directed to the ECU, which will be sent to the same task until its termination.

The shared namespace for an ECU is named `self.shared` and can be associated to an [ECU Task](#). For instance, a task can create a variable named `self.shared.my_data = True`, that other tasks can use. The shared namespace can be used by different commands or tasks, if referring the same ECU. This area is created by *ELM327-emulator* at the first request referring to an ECU (and, if available, the related ECU Task `start()` method is executed). The shared namespace is already active when any task method is run. This shared area is reset by a communication disconnection, or "ATZ", or *reset* command, or expiration of the P3 timer.

The easiest way to configure tasks is to use `Tasks.RETURN.TERMINATE` (so that a task terminates after the execution of the invoked method, e.g., `Task.RETURN.ANSWER(answer)`) and to exploit `self.shared` to store persistent data, shared by different tasks and functions. The easiest way to initialize shared data is through the definition of a `start()` method inside the related ECU Task.

ECU Tasks

An ECU Task is an optional request preprocessor which owns the shared namespace for its related ECU and is executed for each request referred to the same ECU, before interpreting the request (or running the task, regardless the request is a task or a simple command).

The plugin of the ECU Task shall be named "task_ecu_" followed by the hex header digits (case-insensitive) of the (source/destination) CAN id of the ECU (then followed by ".py"). For instance, in case of a request directed to an ECU with CAN id "7E0" (ATSH 7E0), the task ECU plugin shall be named "task_ecu_7E0.py". In case of ATSH 8011F1, the related ECU task shall be named "task_ecu_11F1.py". Hex digits accept the following metacharacters:

- *X* for any single hex digit,
- *W* for any sequence of hex digits (one or more).

The plugin shall implement a class named *Task* derived from the *EcuTasks* class.

Defining an ECU task is not required; if missing, a default shared namespace for the ECU is automatically created when the ECU is first used; no further preprocessing occurs; the task namespace will be anyway usable.

The allowed methods are the same as the normal tasks. The `start()` method of the ECU task is executed upon the first request reference of an ECU, when the shared namespace for the ECU is created. It can for instance be used to run one-time procedures like initializing resources used by subsequent commands (e.g., initialize shared variables and mapped memory used by a task). A typical return code is `return Tasks.RETURN.TASK_CONTINUE(cmd)` (*EcuTask* can be used in place of *Task*), so that the ECU task remains active with its namespace and the request is subsequently processed. `Tasks.RETURN.TERMINATE` can be used for testing purpose.

Example of a plugin implementing an ECU Task which fully disables the ECU processing, always returning "NO DATA" (`"NO DATA"`, `EcuTasks.RETURN.TERMINATE, None`):

```
from elm import EcuTasks

class Task(EcuTasks):
    def run(self, cmd, *_):
        EcuTasks.RETURN.ANSWER("NO DATA")
```



The `run()` method of the ECU is invoked on any request after the first one; if `start()` is not implemented, `run()` is always invoked.

The ECU task is terminated when a method returns with `self.TASK.TERMINATE` (or *False*) in the return tuple, or by the following conditions:

- communication reset (e.g., communication disconnection, or "ATZ", or *reset* command);
- expiration of the P3 timer.

With these two termination conditions, the `stop()` method is also executed (useful for instance to remove login parameters after P3 timer expiration). The default definition of the `stop()` method is to return `Tasks.RETURN.ERROR` (do nothing).

All ECU task methods return the same three-element tuple of the tasks, where the first element is generally *None* (if set to an XML string, its data is written as output response), the second one is either `Tasks.RETURN.CONTINUE` or `Tasks.RETURN.TERMINATE` (the latter is generally only for testing), the third one is the preprocessing output, or *None* for no processing. `TASK_CONTINUE(cmd)` means *None*, `Tasks.RETURN.CONTINUE, cmd`.

The plugin named [task_ecu_11F1.py](#) is an example of ECU Task.

Example

For instance, a plugin named *plugins/task_routine.py* defines the task *task_routine*, which is configured in the dictionary element 'A_ROUTINE', like the following example:

```

'UDS_START_DIAG_SESS': {
    'Request': '^1085' + ELM_FOOTER, # 85 = Flash Programming Session
    'Descr': 'UDS Start Diagnostic Session',
    'Response': PA('') # Response: 50 85
},
'UDS_REQ_SEED': { # Start seed & key and request the seed from ECU
    'Request': '^2701' + ELM_FOOTER,
    'Descr': 'UDS SecurityAccess - requestSeed',
    'Response': PA('12 34') # Response: 67 01 12 44; seed is 12 44
},
'UDS_SEND_KEY': {
    'Request': '^2702' + ELM_DATA_FOOTER,
    'Descr': 'UDS SecurityAccess - Send Key to ECU',
    'Exec': 'self.shared.auth_successful = cmd[4:] == "3322"', # Key
    'Info': '"auth_successful: %s", self.shared.auth_successful',
    'ResponseFooter': lambda self, cmd, pid, uc_val: (
        PA('') if self.shared.auth_successful else NA('35')
    ) # Response: 67 02 if pos.; 7F 27 35 if neg. 35=invalidKey
},
'A_ROUTINE': { # UDS Routine Control (31): Start (01)
    'Request': '^3101' + ELM_DATA_FOOTER,
    'Descr': 'An UDS Routine',
    'Task': 'task_routine'
},

```

The following table shows the UDS protocol sequence for this example:

Application (request)	ECU (response)	Protocol	Description
10 85	50 85	StartDiagnosticSession, ECUProgrammingMode	Start Programming Mode (ECU returns a positive answer)
27 01	67 01 12 44	SecurityAccess, requestSeed	Start seed & key and request seed to ECU (ECU returns a positive answer, the seed 12 44 is then sent back from the ECU to the application)
27 02 33 22	67 02	SecurityAccess, sendKey	Send security key 33 22 to ECU (ECU returns a positive answer)
31 01	71 01	StartRoutineByLocalIdentifier, ID = 01 (Start)	Start flash driver download into RAM (ECU returns a positive answer)

In such example, a request of type 3101... will start the task *task_routine*, which can immediately return (if `Tasks.RETURN.TERMINATE` is used), or (in case `Tasks.RETURN.CONTINUE` is used) will also be able (not in this example of task) to process any subsequent request (also if not matching 3101...), until the plugin is terminated. In the above example, the `'Header'` attribute is not set, so any CAN header will be valid.

The following code shows a sample of 7E0 ECU task related to the Python plugin named "task_ecu_7E0.py"; the `start()` method is executed the first time the ECU is used, while the `stop()` method is executed on expiration of the P3 timer; both reset the login state to `False`:

```

from elm import EcuTasks

# 7E0 ECU task
class Task(EcuTasks):
    def start(self, cmd, *_):
        self.auth_successful = False
        return EcuTasks.RETURN.TASK_CONTINUE(cmd)

    def stop(self, cmd, *_):
        if self.auth_successful:
            self.logging.error('Login timeout')
        self.auth_successful = False
        return EcuTasks.RETURN.TASK_CONTINUE(cmd)

```

Notice that `self.auth_successful` in the ECU task can be used in place of `self.shared.auth_successful` because the ECU task owns the ECU namespace.

The following code shows a basic task related to the Python plugin named "task_routine.py"; the assumption is that the default `self.shared.auth_successful` state (`False`) in the ECU shared namespace is already set by the ECU task:

```
from elm import Tasks

class Task(Tasks): # UDS Routine Control (31): Start (01)
    def run(self, cmd, *_):
        if self.shared.auth_successful:
            self.logging.info('Routine %s successfully executed.',
                              cmd[4:])
            return Task.RETURN.ANSWER(self.PA('')) # Return 71 01
        else:
            self.logging.info('Security Access Denied for routine %s.',
                              cmd[4:])
            return Task.RETURN.ANSWER(self.NA('33')) # Return 7F 31 33
```

In the above example, the shared `self.shared.auth_successful` attribute is checked in order to immediately return either a positive or negative answer (`Task.RETURN.ANSWER` uses `Tasks.RETURN.TERMINATE`). Notice also that the `UDS_SEND_KEY` PID sets `self.shared.auth_successful` to `True` if the security key is correct and returns either positive or negative answer, exploiting a `ResponseFooter` type lambda function. The example shows that both tasks and response functions are ways to implement process steps (in the specific case shown by the example, the result is similar). While response functions are quick to implement, tasks allow much more flexibility.

Notice that `self.shared` is only available within the same ECU. If multiple ECUs are concurrently configured and interacted (each one with its own header), different ECUs will have their own shared namespace.

The `tasks` command returns the dump of all the used namespaces, both ECU-shared and in-task ones.

The plugins named `task_mt05_read_mem_addr.py` and `task_mt05_write_mem_addr.py` show how to read and write memory by address, mapping the memory space into a file.

The plugin named `task_erase_memory.py` shows how to use the `start()` and `run()` methods, as well as `Tasks.RETURN.CONTINUE` which simulates a certain function processing time.

The plugin named [task_ecu_11F1.py](#) is an example of memory map run at the first usage of the 11F1 ECU. The `task_mt05_...` plugins assume that the memory map structures are already instantiated by the ECU task.

Helper functions

Within a task, the helper function `self.task_request_matched(request)` checks whether the request in the argument (typically returned by `self.multiline_request()`) matches the original request that invoked the task. This is because a task might be called more times if remaining active; this function can for instance differentiate a possible `TesterPresent` check (which can be forwarded to the standard processor) from the task request (which can be processed within the task, without forwarding it).

The helper functions `self.HD(header)`, `self.SZ(size)`, `self.DT(data)`, `self.AW(data)`, `self.NA(data)` and `self.PA(data)` support the generation of an XML response, similarly to the functions used in the dictionary.

The helper function `self.task_get_request()` gets the original request command that initiated the task (used with `Tasks.RETURN.CONTINUE` to return the original request while processing consecutive ones).

Available interfaces

ELM327-emulator allows the following interfaces:

- serial communication using a pseudo-terminal, as default mode on non Windows operating systems (without options),
- TCP/IP networking, when using option `-n`, followed by the TCP/IP port,
- serial COM port, when using option `-p`, default mode with Windows (the option is followed by the port name and allows setting a baud rate with the `-a` option),
- standard communication, when using option `-P`.

Usage of a pseudo-terminal

On non Windows operating systems, the default mode (without communication options) creates a pseudo-tty driver to be used to connect client applications. It is shown at startup, can be read by applications in batch mode and the `port` command returns it at any moment.

```
python3 -m elm -s car
```



Usage of a TCP/IP connection

The `-n` options uses a TCP socket; the most commonly used one is 35000.

```
python3 -m elm -s car -n 35000
```



Usage of a serial communication port

The `-p` option exploits the [pyserial](#) library. A baud rate can be set with the `-a` option (defaulted to 38400 bps). With Windows, the COM port is defaulted to COM3.

To open a specific Windows port:

```
python3 -m elm -s car -p COM3
```



To open a specific Windows port (which is the same as `python3 -m elm -s car` because corresponds to default values):

```
python3 -m elm -s car -p COM3 -a 38400
```



Usage of the standard communication option

The `-P` option uses the standard communication mode allowed by the operating system when opening a read/write device, including a special file (for instance, for Bluetooth serial networking with UNIX/Linux).

Usage of Bluetooth with Windows

We need to pair the client device and add an incoming RFCOMM port. Then *ELM327-emulator* can access that virtual COM port with the `-p` option.

With Windows, run "Settings", "Bluetooth & other devices", "More Bluetooth options", "COM ports" tab, press "Add", select "Incoming", press "OK". For instance, this procedure creates a virtual COM4 port.

Then run:

```
python -m elm -p COM4 -s car
```



Note: adding the `-l` option may be needed in some cases and is important when the configuration changes the way command lines are sent, closing lines with a newline instead of a CR.

Usage of Bluetooth with UNIX/Linux

After creating a Bluetooth special file on a UNIX/Linux system implementing an RFCOMM port, *ELM327-emulator* can access that file with the `-P` option.

Install Bluetooth tools and daemons:

```
sudo apt-get install bluez
```



Pairing the devices:

```
bluetoothctl
[bluetooth]# power on
[bluetooth]# agent on
[bluetooth]# default-agent
[bluetooth]# discoverable on
[bluetooth]# pairable on
```



Run the client application, select the Bluetooth device name, then perform pairing on both systems, answering yes.

Note. In case removing or unpairing a paired address is needed:

```
remove <MAC address>
power off
```

After pairing, quit *bluetoothctl*:

```
[bluetooth]# exit
```

Bluetooth preparation:

```
sudo service bluetooth restart
sdptool add --channel=1 SP
rfcomm release 0 # if needed
```

Configure the special file */dev/rfcomm0* (see below):

```
sudo mknod -m 666 /dev/rfcomm0 c 216 0
sudo chown $USER /dev/rfcomm0
```

Run *ELM327-emulator* with bluetooth interface:

```
rfcomm watch /dev/rfcomm0 1 python3 -m elm -P /dev/rfcomm0 -l -s car
```

Note: the `-l` option may be needed in some cases and is important when the Bluetooth configuration changes the way command lines are sent, closing lines with a newline instead of a CR.

The */dev/rfcomm0* device driver can be manually created in order to avoid the error "*Can't open RFCOMM device: Permission denied*" when running *rfcomm* as a standard user (an not as root). First run *rfcomm* as superuser and connect a client:

```
sudo rm -f /dev/rfcomm0; sudo rfcomm listen /dev/rfcomm0 1 ls -l /dev/rfcomm0
```

After a client connection, you should get:

```
crw-rw---- 1 root dialout 216, 0 mar 28 11:17 /dev/rfcomm0
```

Create the special file with the same major_number and minor_number, using 666 permissions and change ownership:

```
sudo mknod -m 666 /dev/rfcomm0 c 216 0
sudo chown $USER /dev/rfcomm0
```

Note: the Bluetooth error "Can't bind RFCOMM socket: Address already in use" means that the `bind()` function used by the command producing the error failed because there is another socket with the same number already bound by a local application. The way to solve this problem is to find the local application binding that socket and terminating it.

Forwarder options

In order to verify and improve its dictionary, *ELM327-emulator* allows acting as a proxy between a software application and a real OBD-II device. Whenever the OBD-II interface provides data to the application which the dictionary does not include, a warning is shown reporting the answer from the OBD-II interface; besides, a related `unknown_<command>_R` element is added to the `counters`, with the last returned answer from the OBD-II interface, to allow subsequent verifications. The dictionary can then be manually edited to align the *ELM327-emulator* behaviour with the answer returned by the OBD-II interface.

To act as a proxy, *ELM327-emulator* can either expose the virtual serial device or the TCP network port to the application; then it connects the OBD-II interface through an internal forwarder component, allowing serial communication or TCP/IP networking.

The selection of the interface exposed to the application is done via the standard options: by default the pseudo-tty device is used; alternatively, the `-n` option allows using a local TCP/IP port (e.g., `-n 35000`).

The OBD-II interface is connected through the `-S` option (serial device) or the `-H` and `-N` ones (TCP/IP host and related port). When using the serial device, the `-B` option allows indicating a specific baud rate (38400 bps by default).

Data read from the OBD-II port are grouped together basing on a timeout parameter (floating point number) which by default is 0.2 seconds and can be tuned with the `-T` option. The higher the number, the more reliant the grouping; anyway, delays produced by high timeout values might compromise the communication quality: if the application does not perform correctly in the forwarder mode (e.g., producing connection drops), is useful to test different timeout periods, like `-T 0.1`.

Example.

- In a window, run a simulated OBD-II interface connected via TCP network: `python3 -m elm -s car -n 20000`. Then optionally set `loglevel debug`.
- In another window, run *ELM327-emulator* configured as forwarder to the local TCP port 20000 and exposing a network port 35000: `python3 -m elm -s car -n 35000 -N 20000 -H localhost`.
- In a third window, run a telnet client: `elnet localhost 35000`. Write `at@1` and press enter. Check the logs in the other windows.
- Close the telnet client. Run an *OBD* application, select Wifi communication, IP address 127.0.0.1, port 35000. Check the logs in the other windows. You should succeed in connecting the application.

Logging and monitoring

Logs are written to *elm.log*, file, rotated to *elm.log.1* and *elm.log.2* when its size reaches 1 MB. [Logging](#) is controlled through the `elm.yaml` file (in the current directory by default). Its path can be set through the `ELM_LOG_CFG` environment variable. This file follows the [Python's builtin logging module format](#) and allows customizing the configuration of the logging process.

The logging level can be dynamically changed through the `loglevel` command.

To read the current log level:

```
loglevel
```



To set log level to debug:

```
loglevel 10
# or
loglevel debug
```



Press TAB to get the autocompletion of the available loglevel values (either number or word).

Alternatively, the logging level can be set through `logging.getLogger().handlers[n].setLevel()`. To check that *console* is the first handler (e.g., `handlers[0]`), run `for n, l in enumerate(logging.getLogger().handlers): print(n, l.name)`. For instance, if *console* refers to the first handler (default settings of the provided `elm.yaml` file), the following commands will change the logging level:

```
logging.getLogger().handlers[0].setLevel(logging.DEBUG)
logging.getLogger().handlers[0].setLevel(logging.INFO)
logging.getLogger().handlers[0].setLevel(logging.WARNING)
logging.getLogger().handlers[0].setLevel(logging.ERROR)
logging.getLogger().handlers[0].setLevel(logging.CRITICAL)
```



It is possible to add marks in the log file via commands like `logging.info("my mark")`

To totally disable logging for all handlers: `logging.disable(logging.CRITICAL)`. To restore logging: `logging.disable(0)`.

To browse the log files, [lnav](#) is suggested.

Command to count the number of different PIDs (OBD Commands) used by the client (excluding AT Commands):

```
reduce(lambda x, key: x + (1 if re.match('^[A-Z]', key) and not key.startswith('AT_') and emulator.counters[key] > 0
```



The following command returns the total number of OBD Commands (PID queries issued by the client excluding AT Commands):

```
reduce(lambda x, key: x + (emulator.counters[key] if re.match('^[A-Z]', key) and not key.startswith('AT_') else 0), €,
```



To only count AT Commands:

```
reduce(lambda x, key: x + (emulator.counters[key] if key.startswith('AT_') else 0), emulator.counters, 0)
```



Print the average number of processed commands per second within a 5 seconds period:

```
test at@1 # add at least one command
a=emulator.counters['commands'];time.sleep(5);print((emulator.counters['commands']-a)/5)
```



Same as before, printing the average number of processed commands per second within a 1 second period for 20 times:

```
test at@1 # add at least one command
for i in range(20): \n\t a=emulator.counters['commands'] \n\t time.sleep(1) \n\t print((emulator.counters['commands']-
```



To save a CSV file including the *emulator.counters* dictionary:

```
with open('mycounters.csv', 'w') as f: f.write('\r\n'.join([x + ', ' + repr(emulator.counters[x]) for x in emulator.c
```



ObdMessage Dictionary Generator for "ELM327-emulator" (obd_dictionary)

obd_dictionary is a dictionary generator for "ELM327-emulator".

It queries the vehicle via *python-OBD* for all available commands and is also able to process custom PIDs described in [Torque CSV files](#).

Its output is a Python *ObdMessage* dictionary that can either replace the *obd_message.py* module of *ELM327-emulator*, or extend the existing dictionary via *merge* command, so that the emulator will be able to provide the same commands returned by the vehicle.

Notice that querying the vehicle might be invasive and some commands can change the car configuration (enabling or disabling belts alarm, enabling or disabling reverse beeps, clearing diagnostic codes, controlling fans, etc.). In order to prevent dangerous PIDs to be used for building the dictionary, a PID blacklist (*blacklisted_pids*) can be edited in [elm.py](#). To check all PIDs without performing actual OBD-II queries (dry-run mode), use the `-p 0` option (the standard error output with default logging level shows the list of produced PIDs).

obd_dictionary can be run as:

```
python3 -m obd_dictionary --help
```



or simply:

```
obd_dictionary --help
```



Command line arguments:

```
usage: obd_dictionary [-h] -i DEVICE [-c CSV_FILE] [-o FILE] [-v] [-V] [-p PROBES] [-B BAUDRATE]
                    [-T TIMEOUT] [-C] [-F] [-P PROTOCOL] [-d DELAY] [-D DELAY_COMMANDS] [-n CAR_NAME]
                    [-b] [-r] [-x] [-t [FILE]] [-m]
```

optional arguments:

- h, --help show this help message and exit
- i DEVICE python-OBd interface: serial port connected to the ELM327 adapter (required argument).
- c CSV_FILE, --csv CSV_FILE input csv file including custom PIDs (Torque CSV Format: <https://torque-bhp.com/wiki/PIDs>) '-' reads data from the standard input
- o FILE, --out FILE output dictionary file generated after processing input data (replaced if existing). Default is to print data to the standard output
- v, --verbosity print process information
- V, --verbosity_debug print debug information
- p PROBES, --probes PROBES number of probes (each probe includes querying all PIDs to the OBD-II adapter)
- B BAUDRATE, --baudrate BAUDRATE python-OBd interface: baudrate at which to set the serial connection.
- T TIMEOUT, --timeout TIMEOUT python-OBd interface: specifies the connection timeout in seconds.
- C, --no_check_voltage python-OBd interface: skip detection of the car supply voltage.
- F, --fast python-OBd interface: allows command optimization (CR to repeat, response limit).
- P PROTOCOL, --protocol PROTOCOL python-OBd interface: forces using the given protocol when communicating with the adapter.
- d DELAY, --delay DELAY delay (in seconds) between probes
- D DELAY_COMMANDS, --delay_commands DELAY_COMMANDS delay (in seconds) between each PID query within all probes
- n CAR_NAME, --name CAR_NAME name of the car (dictionary label; default is "car")
- b, --blacklist include blacklisted PIDs within probes
- r, --dry-run test the python-OBd interface in debug mode.
- x, --noautopid do not autopopulate the pid list with the set of built-in commands supported by the vehicle; only use csv file.
- t [FILE], --at [FILE] include AT Commands within probes. If a dictionary file is given, also extract AT Commands from the input file and add them to the output
- m, --missing add in-line comment to dictionary for PIDs with missing response

ObdMessage Dictionary Generator for "ELM327-emulator".

Sample usage: `obd_dictionary -i /dev/ttyUSB0 -c car.csv -o AurisOutput.py -v -p 10 -d 1 -n mycar`

`obd_dictionary` exploits the [command discovery feature](#) of `python-OBd`, which autopopulates the set of builtin commands supported by the vehicle through [queries](#) performed within the [connection phase](#). Optionally, this set can be further enriched with a list of custom PIDs included in an input csv file in [Torque CSV Format](#). The autopopulation feature can be disabled with `-x` option.

The command allows all the python-OBd interface settings (see `-B`, `-T`, `-C`, `-F`, `-P` command-line options) and a dry-run flag (`-r`), which is very useful to test the OBD-II connection.

For instance, the following command tests an OBD-II connection via Bluetooth using the [related recommendations](#) described in the python-OBd repository.

```
python3 -m obd_dictionary -i /dev/rfcomm0 -B 38400 -T 30 -r
```

See also [this post](#) for Bluetooth.

For better analysis, the `-r` output can be piped to [lnav](#) (the following command tests the USB connection):

```
python3 -m obd_dictionary -i /dev/ttyUSB0 -B 38400 -r 2>&1 | lnav
```

When the tests provide successful connection, the `-r` option can be removed and the additional `obd_dictionary` options can be added.

obd_dictionary can be also used to test *elm*. Run `python3 -m elm -s car`. Read the pseudo-tty, say `/dev/pts/2` (this mode uses the serial communication). Run *obd_dictionary*:

```
python3 -m obd_dictionary -i /dev/pts/2 2>&1 | lnav
```



(The automation of this process is shown further on.)

In general, *ELM327-emulator* should already manage all needed AT Commands within its default dictionary, so in most cases it is worthwhile removing them from the new scenario via `-t` option.

The file produced by *obd_dictionary* provides the same information model of *obd_message.py*. It can be used to replace the default module or can be dynamically imported in *ELM327-emulator* through the `merge` command, which loads an *ObdMessage* dictionary and merges it to *emulator.ObdMessage*. Example of *merge* process:

```
# Create AurisOutput.py
obd_dictionary -i /dev/ttyUSB0 -c auris.csv -o AurisOutput.py -n Auris
python3 -m elm # run ELM327-emulator
merge AurisOutput
scenario Auris
```



To help to configure the emulator, autocompletion is allowed (by pressing TAB) when prompting the `merge` command, including the `merge` argument. Also variables and keywords like `scenario` accept autocompletion, including the `scenario` argument.

A merged scenario can be removed via `del emulator.ObdMessage['<name of the scenario to be removed>']`.

To produce a complete dictionary file that can replace *obd_dictionary*:

```
obd_dictionary -i /dev/ttyUSB0 -c auris.csv -o AurisOutput.py -n default -t elm/obd_message.py
```



ELM327-emulator batch mode

ELM327-emulator can be run in batch mode to allow automating tests and background execution. The `-b FILE` option allows this mode and writes the output to FILE. When using serial communication, the first line in that file will be the virtual serial device, which can be read to a shell variable through `read variable_name < output_file`. Commands can be piped in (e.g., within a bash script) to configure the emulator (e.g., via `echo -e`). The appropriate way to kill a background instance of the emulator is with the SIGINT signal (`kill -2`). To ensure that the external application is started only after correct setup of the emulator, the input commands can be terminated with a string (e.g., "RUNNING") that can then be recognised before starting the application.

elm offers four operation modes:

- interactive (providing a command line prompt). This is activated by default, when neither `-b` option nor `-d` is used;
- batch mode with input commands, activated when the `-b` option (`-b file` or `-b -` e.g., `-b -` for standard log output, or `-b output_file_name`). This mode reads the standard input for the same commands that can be issued by the user in interactive mode;
- batch mode light, only available with UNIX/Linux, without input commands (no separate thread is created and the command interpreter is not used). This is activated when both `-b` and `-d` options are used;
- daemon mode without input commands, only available with UNIX/Linux, allowing starting and stop the process in UNIX/Linux daemon mode, with `-d` option (start) and `-t` (terminate).

In daemon mode, a lock file is used to prevent multiple instances.

Note: with Windows, options `-d` and `-t` are not available.

The following script shows an example of batch mode usage. *obd_dictionary* is run after starting *ELM327-emulator* in background and is used here as example of external application interfacing the emulator. The output of the emulator is saved to \$FILE and the background process id is saved to \$EMUL_PID.

```
#!/usr/bin/env bash

set -o errexit
set -o pipefail
set -o nounset

FILE=/tmp/elm$$
echo -e 'scenario car\ncounters' | elm -b "${FILE}" &
EMUL_PID=$!

until [ ! -f "${FILE}" ] || grep -q "End of batch commands." "${FILE}"
do sleep 0.5
done
read TTYNAME < "${FILE}"

obd_dictionary -i "${TTYNAME}" -t -v -o /dev/null

kill -INT "${EMUL_PID}"
cat "${FILE}"
rm "${FILE}"
```



Python API

Instantiating the class

All arguments are optional.

```
from elm import Elm

emulator = Elm(
    batch_mode=False,          # optional flag to indicate different logging for batch mode
    newline=False,            # optional flag to use newline (<NL>) instead of carriage return <CR> for detecting :
    serial_port="",           # optional serial port used with Windows (ignored with non Windows O.S.)
    serial_baudrate="",       # baud rate used by any serial port (but the forward port); default is 38400 bps
    net_port=None,            # number for the optional TCP/IP network port, alternative to serial_port
    forward_net_host=None,    # host used when forwarding the client interaction to a remote OBD-II device
    forward_net_port=None,    # port used when forwarding the client interaction to a remote OBD-II device
    forward_serial_port=None, # serial port name when forwarding the client interaction to an OBD-II device via sei
    forward_serial_baudrate = None, # used baud rate for the forwarded serial port; default is 38400 bps
    forward_timeout=None)     # floating point number indicating the read timeout when configuring a forwarded OBD-
```



`get_pty()` returns the used port.

Interactive mode

Interactive mode uses the Context Manager:

```
from elm import Elm
import time

with Elm() as session:
    # interactive monitoring
    pty = session.get_pty()
    print(f"Used port: {pty}")
    time.sleep(40) # example
```



Example of TCP/IP network usage:

```
from elm import Elm
import time

with Elm(net_port=35000) as session:
    time.sleep(40) # example
```



Batch/daemon mode

Batch mode without interaction does not need the Context Manager:

```
from elm import Elm

emulator = Elm(batch_mode=True)

pty = emulator.get_pty()
print(f"Used port: {pty}")

emulator.run()
```



Software architecture

When using the Context Manager, a thread is started and the current context is returned to the user. The created thread opens a bidirectional pty-type pipe and processes the related I/O.

When not using the Context Manager, no background thread is created and the pipe is run in the current context.

Testing OBD-II applications

Simple testing

With UNIX/Linux, the serial communication can be tested with *screen*.

```
python3 -m elm
```



In another terminal:

```
sudo apt-get install screen
screen /dev/pts/3
```



The TCP/IP networking can be tested via *telnet*.

```
python3 -m elm -n 35000
```



In another terminal:

```
sudo apt-get install telnet
telnet localhost 35000
```



python-OBD

[python-OBD](#) is a Python module for handling realtime sensor data from OBD-II vehicle ports supporting ELM327 OBD-II adapters. *obd_dictionary* (which internally exploits python-OBD) can be used to test it. For instance, with Linux:

- open a terminal and run `python3 -m elm -s car` (read the returned pty port)
- open another terminal and run `python3 -m obd_dictionary -i /dev/pts/0 | less` (use the same port returned by *ELM327-emulator*)

OBD Auto Doctor

One of the applications which can be used to test *ELM327-emulator* is [OBD Auto Doctor](#). It supports different operating systems, including Windows, Mac, Linux, Android, iOS and enables communicating with OBD-II to get summary information, trouble codes, advanced diagnostics, real time graphical monitoring and many other in-depth data on the ECUs.

The following instructions explain how to use it with Ubuntu for testing *ELM327-emulator*:

Download the Linux application from the [Obdautodoctor site](https://github.com/Ircama/ELM327-emulator). Check prerequisites.

Install with `sudo dpkg -i obd-auto-doctor....deb`.

Run *ELM327-emulator* and select the *car* scenario:

```
python -m elm -s car
```

This mode uses the serial communication. Copy the pseudo-tty device reported by *ELM327-emulator*.

Run *OBD Auto Doctor* with `obdautodoctor`. Then select File, Open connection, Connection method: Serial port. Use manual settings. Paste the pseudo-tty device in the COM port box. Press Connect.

OBD Auto Doctor also supports TCP/IP communication. Run *ELM327-emulator* using TCP/IP networking:

```
python -m elm -s car -n 35000
```

Run *OBD Auto Doctor* with `obdautodoctor`. Then select File, Open connection, Connection method: WiFi. IP address: 127.0.0.1. Port: 35000. Press Connect.

Scantool

Scantool from [ScanTool.net](https://github.com/Ircama/scantool) is an old software which can also be used to test *ELM327-emulator*.

Recent repository: <https://github.com/kees/scantool>

Software ported to Ubuntu 20.04 LTS: https://github.com/Ircama/scantool/tree/pts_support

Installation:

```
git clone --branch pts_support https://github.com/Ircama/scantool.git
sudo apt install liballegro4.4 liballegro4-dev allegro4-doc
make clean
make -e RELEASE=yes LOG=yes
```

To run the application: `./scantool`

This version of *scantool* allows configuring a pseudo-tty support by editing `~/scantoolrc`. See related *readme.txt*. With Ubuntu, this can be automated by *ELM327-emulator* via the following plugin:

```
import fileinput
import os
import re

def scantool(port):
    numport = str(int(os.path.basename(port)) + 1000)
    with fileinput.FileInput(os.path.expanduser("~/scantoolrc"), inplace=True) as file:
        for line in file:
            match = re.sub(r"^comport_number *=.*", "comport_number = " + numport, line)
            if match:
                print(match, end='')
            else:
                print(line, end='')

```

Close *scantool*, save a file named *scantool.py* including the above reported plugin. Run *ELM327-emulator*:

```
python3 -m elm -s car
from scantool import scantool;scantool(emulator.slave_name) # load and run the plugin
```

To run the application integrated with *ELM327-emulator*: `./scantool`

HUD ECU Hacker

[HUD ECU Hacker](#) is a great application developed by [ElmüSoft](#).

It is an OBD-II scanner software specialized to manage ECU's from Delphi Electronics, including flash memory download and upload functions. The application runs on Windows and is very well engineered, extensively using OBD-II and UDS, with wide set of functionalities and robust frame control handling.

ELM327-emulator is already able to provide a basic emulation of the Delphi MT05 ECU and, if needed, can be extended via the development of additional tasks and through the editing of the ObdMessage configuration.

C program sample

Releases

When using a C program to connect to the *ELM327-emulator*, you can optionally configure a raw terminal. Connecting to the *ELM327-emulator* should look similar to below (error checking ommitted).

v3.0.5

on Dec 12

+ 2 releases

Deployme

github-

+ 4 deployr

Packages

No packages

Contribut



Language:

Python 9

```
/* Open the device */
fd = open("/dev/pts/...", O_RDWR | O_NOCTTY | O_SYNC);
...

/* Optionally, you could configure the terminal in row mode (not required) */
struct termios tty;
cfmakeraw(&tty);
...

/* Set 38400, 8N1 */
tty.c_cflag &= ~CSIZE; /* clear size */
tty.c_cflag |= CS8; /* 8-bit size */
tty.c_cflag &= ~PARENB; /* no parity */
tty.c_cflag &= ~CSTOPB; /* 1 stop bit */

cfsetospeed(&tty, B38400);
cfsetispeed(&tty, B38400);

/* Update attributes */
rc = tcflush(fd, TCIFLUSH);
rc = tcsetattr(fd, TCSANOW, &tty);

/* Reset the device */
rc = write(fd, "ATZ\r", 4);
```

Running on Windows

When natively running on Windows (to be used when connecting a Windows application), *ELM327-emulator* requires a virtual serial port driver providing a virtual COM port pair (like *com0com*), so that one COM port (e.g., COM4) can be used to connect the application and the other one (e.g., COM3) the *ELM327-emulator*. By default, *ELM327-emulator* uses the COM3 serial port; any other port can be set through the `-p` argument. Example:

```
python3 -m elm -p COM5
```

Standards

- The UDS Application layer is reported in [ISO 14229-1:2020](#) (former ISO 15765-3, UDS on CAN)
- [ISO 15765-2](#) (transport protocol and network layer services) describes the CAN protocol
- [ISO 15765-3:2004](#) describes the implementation of unified diagnostic services (UDS on CAN at the Session and Application Layer)
- [ISO 14229-2:2013](#): UDS Session layer services
- [ISO 14230-2:1999](#): Keyword Protocol 2000 Data Link Layer
- [ISO 14230-3:1999](#): Keyword Protocol 2000 Application Layer
- OBD-II pids: SAE J1979 E/E Diagnostic Test Modes / ISO 15031

Credits

Thanks to [@qqj1228](#) for implementing support to [com0com Windows driver](#).

Thanks to ElmüSoft for several clarifications and to [mickeyl](#) for some notes on UDS.

License

(C) Ircama 2021 - [CC BY-NC-SA 4.0](#)
